

---

# **Guide Essentiel pour Apprendre Java**

Les fondamentaux du langage : syntaxe, structures de contrôle, programmation orientée objet, collections et bonnes pratiques

Rapport pédagogique — 8 pages

---

# 1. Introduction à Java

Java est un langage de programmation orienté objet, créé par Sun Microsystems en 1995 (aujourd'hui développé par Oracle). Il est réputé pour sa portabilité grâce à sa devise « *Write Once, Run Anywhere* » : un même programme Java peut s'exécuter sur n'importe quelle plateforme disposant d'une machine virtuelle Java (JVM).

## Pourquoi apprendre Java ?

- Très utilisé en entreprise : applications web, mobiles (Android), systèmes embarqués, big data.
- Langage fortement typé, ce qui aide à détecter les erreurs avant l'exécution.
- Écosystème riche : des milliers de bibliothèques et frameworks (Spring, Hibernate...).
- Bonne base pour apprendre d'autres langages orientés objet (C#, Kotlin...).

## Les trois piliers : JDK, JRE et JVM

|            |                                                                                                                     |
|------------|---------------------------------------------------------------------------------------------------------------------|
| <b>JVM</b> | Java Virtual Machine — exécute le bytecode Java, indépendamment du système d'exploitation.                          |
| <b>JRE</b> | Java Runtime Environment — contient la JVM et les bibliothèques nécessaires pour <b>exécuter</b> un programme.      |
| <b>JDK</b> | Java Development Kit — contient le JRE plus les outils pour <b>développer</b> (compilateur javac, débogueur, etc.). |

*Pour développer en Java, il faut installer le JDK (par exemple Java 21, version LTS récente).*

## Compilation et exécution

Le code source Java (fichiers `.java`) est d'abord compilé par `javac` en bytecode (fichiers `.class`), qui est ensuite interprété/exécuté par la JVM avec la commande `java`.

```
javac MonProgramme.java    // compile MonProgramme.java -> MonProgramme.class
java MonProgramme          // exécute le bytecode sur la JVM
```

## 2. Premier programme et structure de base

Tout programme Java commence par une classe. Le point d'entrée est toujours la méthode `main`, qui doit avoir exactement cette signature :

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Bonjour, Java !");  
    }  
}
```

- **public class** : le nom de la classe doit correspondre au nom du fichier (HelloWorld.java).
- **public static void main** : point d'entrée du programme, appelé automatiquement.
- **System.out.println** : affiche du texte suivi d'un retour à la ligne dans la console.

## 3. Variables et types de données

Java est un langage **fortement typé** : chaque variable doit avoir un type déclaré. Il existe 8 types primitifs, ainsi que des types objets (String, tableaux, classes...).

|                                                                                                                                                                                                                                         |                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>int</b>                                                                                                                                                                                                                              | Nombre entier (32 bits). Ex : <code>int age = 25;</code>                                 |
| <b>double</b>                                                                                                                                                                                                                           | Nombre décimal (64 bits). Ex : <code>double prix = 19.99;</code>                         |
| <b>boolean</b>                                                                                                                                                                                                                          | Vrai ou faux. Ex : <code>boolean actif = true;</code>                                    |
| <b>char</b>                                                                                                                                                                                                                             | Un seul caractère. Ex : <code>char lettre = 'A';</code>                                  |
| <b>long / short / byte</b>                                                                                                                                                                                                              | Variante d'entiers de tailles différentes.                                               |
| <b>float</b>                                                                                                                                                                                                                            | Décimal simple précision (moins utilisé que double).                                     |
| <b>String</b>                                                                                                                                                                                                                           | Chaîne de caractères (type objet, pas primitif). Ex : <code>String nom = "Alice";</code> |
| <pre>int age = 25;<br/><br/>double prix = 19.99;<br/><br/>boolean estMajeur = true;<br/><br/>char initiale = 'A';<br/><br/>String prenom = "Camille";<br/><br/>final double TVA = 0.20;    // "final" = constante, non modifiable</pre> |                                                                                          |

## 4. Opérateurs et structures de contrôle

### Opérateurs courants

- Arithmétiques : + - \* / %
- Comparaison : == != < > <= >=
- Logiques : && (et) || (ou) ! (non)
- Affectation : = += -= \*= /=

### Conditions : if / else / switch

```
int note = 14;

if (note >= 16) {
    System.out.println("Très bien");
} else if (note >= 10) {
    System.out.println("Admis");
} else {
    System.out.println("Insuffisant");
}

// switch (depuis Java 14, syntaxe simplifiée)
String jour = switch (3) {
    case 1 -> "Lundi";
    case 2 -> "Mardi";
    case 3 -> "Mercredi";
    default -> "Inconnu";
};
```

### Boucles : for, while, do-while

```
for (int i = 0; i < 5; i++) {
    System.out.println("i = " + i);
}

int compteur = 0;
while (compteur < 3) {
    System.out.println("Compteur : " + compteur);
    compteur++;
}

// boucle "for-each" pour parcourir une collection ou un tableau
int[] notes = {12, 15, 9};
for (int n : notes) {
    System.out.println(n);
}
```

## 5. Tableaux et méthodes

### Tableaux (arrays)

Un tableau contient un nombre fixe d'éléments du même type, indexés à partir de 0.

```
int[] notes = {12, 15, 9, 18};

System.out.println(notes[0]);    // 12

System.out.println(notes.length); // 4


int[][] matrice = new int[3][3]; // tableau 2D (3x3)
```

### Méthodes (fonctions)

Une méthode encapsule un bloc de code réutilisable. Elle possède un type de retour, un nom, et éventuellement des paramètres.

```
public class Calculatrice {

    // Méthode qui retourne un résultat
    static int addition(int a, int b) {
        return a + b;
    }

    // Méthode sans retour (void)
    static void afficherMessage(String message) {
        System.out.println("Message : " + message);
    }

    public static void main(String[] args) {
        int resultat = addition(4, 7);
        afficherMessage("Le résultat est " + resultat);
    }
}
```

Le mot-clé 'static' signifie que la méthode appartient à la classe elle-même, et non à une instance particulière — nous verrons cette distinction en détail page suivante.

## 6. Programmation orientée objet (POO)

La POO organise le code autour d'**objets**, instances de **classes**, qui combinent données (attributs) et comportements (méthodes).

### Déclarer une classe et créer des objets

```
public class Voiture {  
    // Attributs (état de l'objet)  
    private String marque;  
    private int vitesse;  
  
    // Constructeur  
    public Voiture(String marque) {  
        this.marque = marque;  
        this.vitesse = 0;  
    }  
  
    // Méthodes (comportement)  
    public void accelerer(int increment) {  
        this.vitesse += increment;  
    }  
  
    public String getMarque() {  
        return marque;  
    }  
}  
  
// Utilisation :  
Voiture maVoiture = new Voiture("Toyota");  
maVoiture.accelerer(50);  
System.out.println(maVoiture.getMarque()); // "Toyota"
```

### Encapsulation

Les attributs sont souvent déclarés `private` pour empêcher un accès direct depuis l'extérieur ; on y accède via des méthodes publiques (*getters* et *setters*), ce qui protège la cohérence des données.

|                  |                                                            |
|------------------|------------------------------------------------------------|
| <b>private</b>   | Accessible uniquement dans la classe.                      |
| <b>public</b>    | Accessible depuis n'importe où.                            |
| <b>protected</b> | Accessible dans la classe, le package et les sous-classes. |

## 7. Héritage, interfaces et exceptions

### Héritage et polymorphisme

Une classe peut hériter des attributs et méthodes d'une autre grâce à `extends`. Le polymorphisme permet à une sous-classe de redéfinir (*override*) le comportement d'une méthode héritée.

```
class Animal {  
    void crier() {  
        System.out.println("Un animal fait du bruit");  
    }  
}  
  
class Chien extends Animal {  
    @Override  
    void crier() {  
        System.out.println("Le chien aboie");  
    }  
}  
  
Animal monAnimal = new Chien();  
monAnimal.crier();    // "Le chien aboie" (polymorphisme)
```

### Interfaces

Une interface définit un contrat (des méthodes que la classe doit implémenter), sans imposer d'implémentation. Elle permet de simuler l'héritage multiple.

```
interface Volant {  
    void voler();  
}  
  
class Oiseau implements Volant {  
    public void voler() {  
        System.out.println("L'oiseau vole");  
    }  
}
```

### Gestion des exceptions

Java permet de gérer les erreurs à l'exécution avec les blocs `try` / `catch` / `finally`, évitant que le programme ne s'arrête brutalement.

```
try {  
    int resultat = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Erreur : division par zéro !");  
} finally {  
    System.out.println("Bloc exécuté dans tous les cas");  
}
```

## 8. Collections et pour aller plus loin

### Le framework Collections

Le package `java.util` fournit des structures de données dynamiques bien plus flexibles que les tableaux classiques.

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| <b>ArrayList</b>  | Liste dynamique ordonnée, avec doublons possibles.    |
| <b>HashSet</b>    | Ensemble non ordonné, sans doublons.                  |
| <b>HashMap</b>    | Association clé → valeur (dictionnaire).              |
| <b>LinkedList</b> | Liste chaînée, efficace pour insertions/suppressions. |

```
import java.util.ArrayList;
import java.util.HashMap;

ArrayList<String> fruits = new ArrayList<>();
fruits.add("Pomme");
fruits.add("Banane");
System.out.println(fruits.get(0)); // "Pomme"

HashMap<String, Integer> ages = new HashMap<>();
ages.put("Alice", 30);
ages.put("Bob", 25);
System.out.println(ages.get("Alice")); // 30
```

### Bonnes pratiques

- Nommer les classes en PascalCase et les variables/méthodes en camelCase.
- Toujours fermer les ressources (fichiers, flux) — utiliser 'try-with-resources'.
- Écrire du code lisible plutôt que trop concis : la maintenabilité prime.
- Utiliser un IDE (IntelliJ IDEA, Eclipse, VS Code) pour l'autocomplétion et le débogage.

### Pour aller plus loin

- Documentation officielle Oracle : [docs.oracle.com/en/java](https://docs.oracle.com/en/java)
- Approfondir les génériques, les flux (Stream API), la programmation fonctionnelle (lambdas).
- Découvrir un framework comme Spring Boot pour le développement web.
- Pratiquer sur des exercices (HackerRank, Codingame, projets personnels).

---

*Fin du guide — bon apprentissage de Java !*